

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Cours complet — Architecture logicielle & systèmes distribués

Objectifs pédagogiques

Comprendre, concevoir et faire évoluer une architecture logicielle, du monolithe propre vers des systèmes distribués résilients, observables et gouvernés.

Temps estimé

6 jours de formation + 3 TP +
exercices + live coding

Prérequis

Bases Java, programmation orientée objet, HTTP, SQL, Spring Boot débutant-intermédiaire

Table des matières

- **Vue d'ensemble**
- **Planning 6 jours**
- **Projet fil rouge**
- **Évaluation**

- [Ressources](#)

Vue d'ensemble

Ce support est conçu comme un vrai cours d'enseignement supérieur : progression du simple vers le complexe, nombreux exemples, TP dépendants, corrigés, live coding et dossier d'architecture.

Définition simple

L'architecture logicielle, c'est la manière d'organiser un système pour qu'il reste utile, maintenable, fiable et compréhensible dans le temps.

Mode enfant

Construire un logiciel, c'est comme construire une ville. On ne place pas la boulangerie, l'hôpital et l'aéroport au hasard. On pense circulation, sécurité, maintenance et croissance.

Mode pro

Une architecture est un ensemble de décisions structurantes : découpage, responsabilité, flux de données, contraintes non fonctionnelles, patterns techniques et gouvernance. Elle oriente le coût d'évolution d'un produit pendant des années.

Jour 1

Fondamentaux, qualités logicielles, couches, monolithe vs distribué.

Jour 2

Patterns d'architecture, DDD de base, ports/adapters, dossiers d'architecture.

Jour 3

Jour 4

Systèmes distribués, CAP, REST, gRPC, messaging, versioning.

Microservices, gateway, discovery, sagas, cohérence éventuelle.

Jour 5

Résilience, observabilité, sécurité, secrets, Zero Trust.

Jour 6

Docker, Kubernetes, CI/CD, ADR, urbanisation, choix d'architecture.

Planning 6 jours

Jour	Thème	Livrable pédagogique	Lien
1	Fondamentaux d'architecture	Cartographie des qualités et premières couches	Ouvrir
2	Patterns et structuration	Choix architectural argumenté + note d'architecture	Ouvrir
3	Systèmes distribués & API	Analyse des compromis et des modes de communication	Ouvrir
4	Microservices	Découpage progressif du projet fil rouge	Ouvrir
5	Résilience & sécurité	Architecture fiabilisée et observable	Ouvrir
6	Cloud & gouvernance	Choix final d'architecture et ADR	Ouvrir

Projet fil rouge

Le fil rouge du cours est une **plateforme de réservation de livres et de prêts universitaires** appelée *Campus Library Flow*. Ce projet a été choisi car il permet de montrer :

- un domaine compréhensible par tous ;
- des cas métier simples au départ ;
- une évolution naturelle vers le distribué : catalogue, emprunts, notifications, authentification, audit ;
- des besoins non fonctionnels réels : disponibilité, sécurité, traçabilité, montée en charge, observabilité.

Cas d'usage entreprise

Une université veut moderniser sa plateforme. Au début, une seule équipe développe un monolithe. Deux ans plus tard, plusieurs services doivent évoluer à des rythmes différents : catalogue, comptes, notifications, analytics, paiements d'amendes. L'architecture doit alors changer.

Évaluation et conseils enseignant

Ce qu'un enseignant peut évaluer

- Qualité du raisonnement architectural
- Justification des compromis
- Structure des couches et séparation des responsabilités
- Qualité API et gestion des erreurs
- Compréhension des limites du distribué
- Qualité du dossier d'architecture et des ADR

Pièges fréquents

- Choisir des microservices "par mode"
- Confondre diagramme et architecture
- Négliger les besoins non fonctionnels
- Créer trop de couches sans valeur métier
- Utiliser des transactions distribuées partout

Ressources du support

- [TP1 — Base architecturale propre](#)
- [TP2 — Passage en distribué](#)
- [TP3 — Résilience, observabilité, sécurité](#)
- [Scénarios de live programming](#)
- [Comment produire un dossier d'architecture](#)
- [Glossaire détaillé](#)
- [FAQ](#)

[Introduction →](#)